

Unit-03 Arrays

- An array is a fixed-size sequential collection of elements of same data types that share a common name.
- It is simply a group of data types.
- An array is a derived data type.
- An array is used to represent a list of numbers , or a list of names.

- For Example :

1. List of employees in an organization.
2. Test scores of a class of students.
3. List of customers and their telephone numbers.
4. List of students in the college.

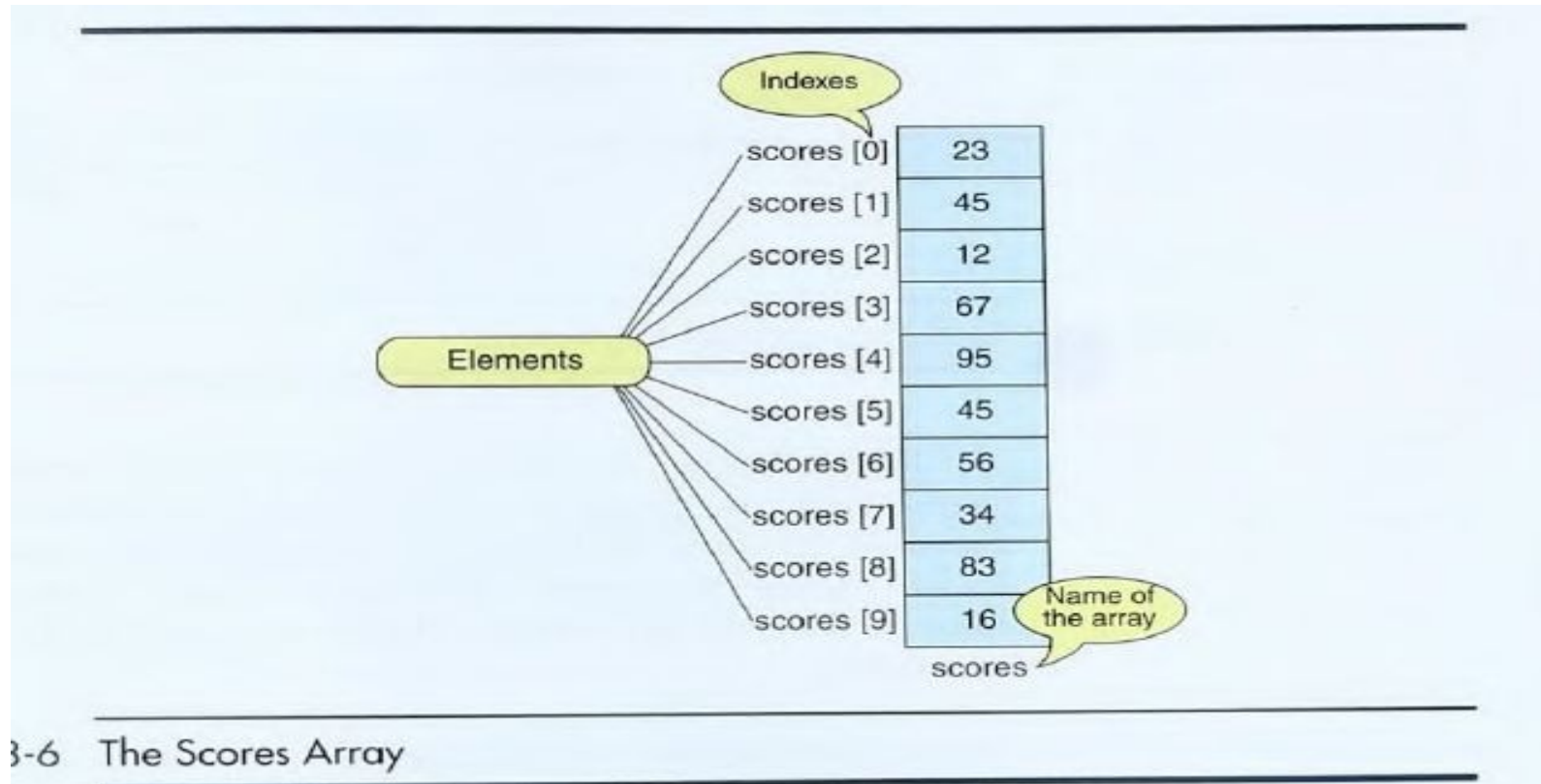
- For Example, to represent 100 students in college , can be written as

student [100]

- Here student is a array name and [100] is called index or subscript.

Unit-3

Using Arrays



Unit-3

Definition and Declaration

```
type arrayName [arraySize]
```

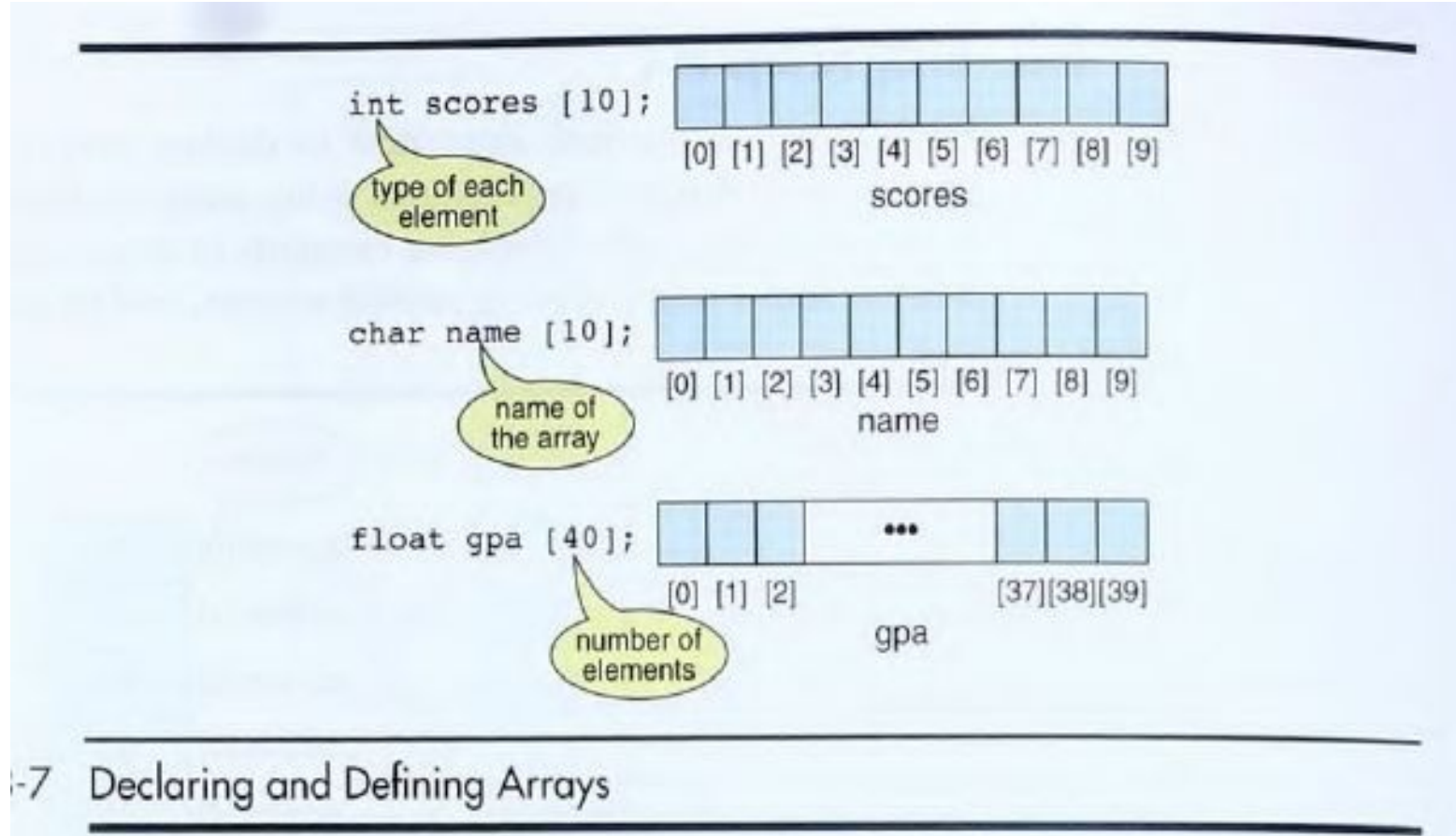
```
data_type array_name [size];
```

or

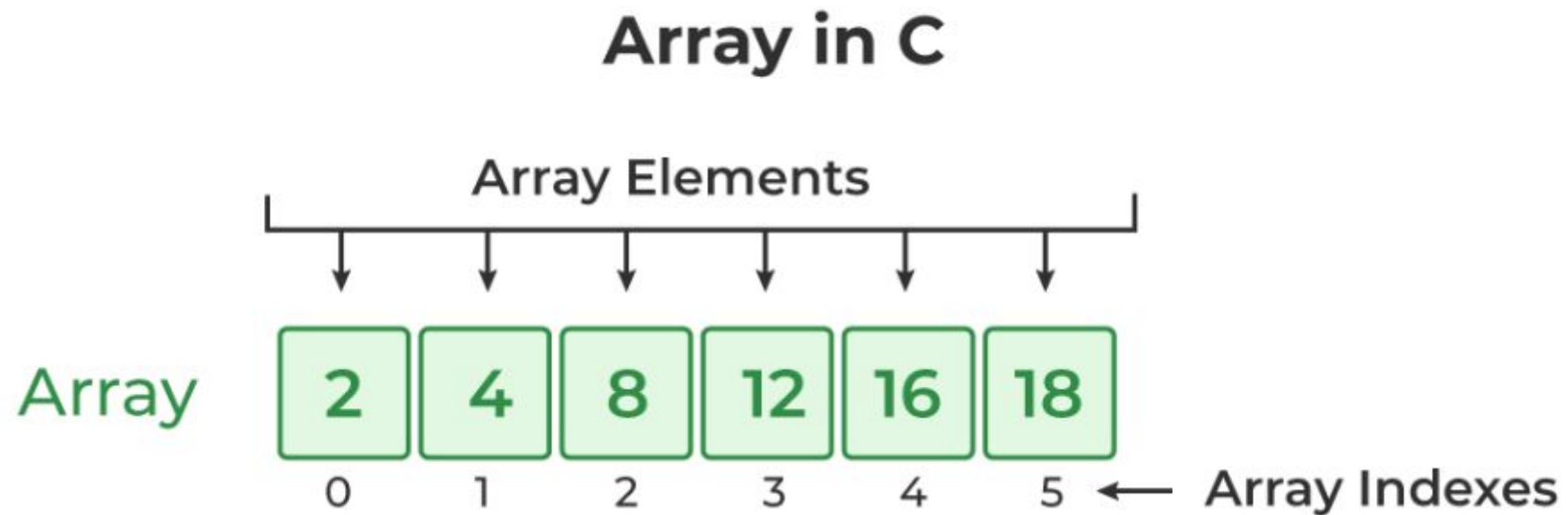
```
data_type array_name [size1] [size2]...[sizeN];
```

Unit-3

Definition and Declaration

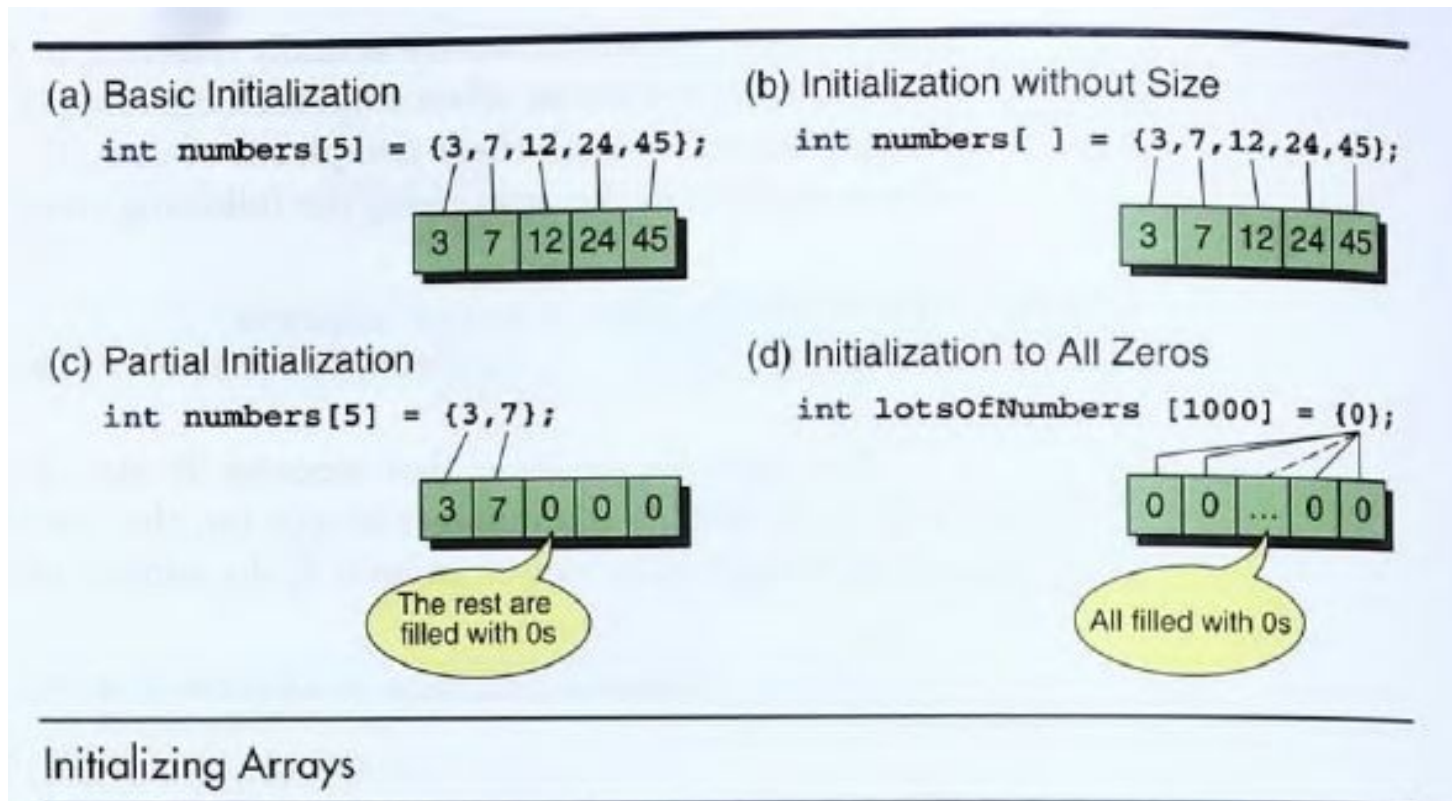


Arrays in C



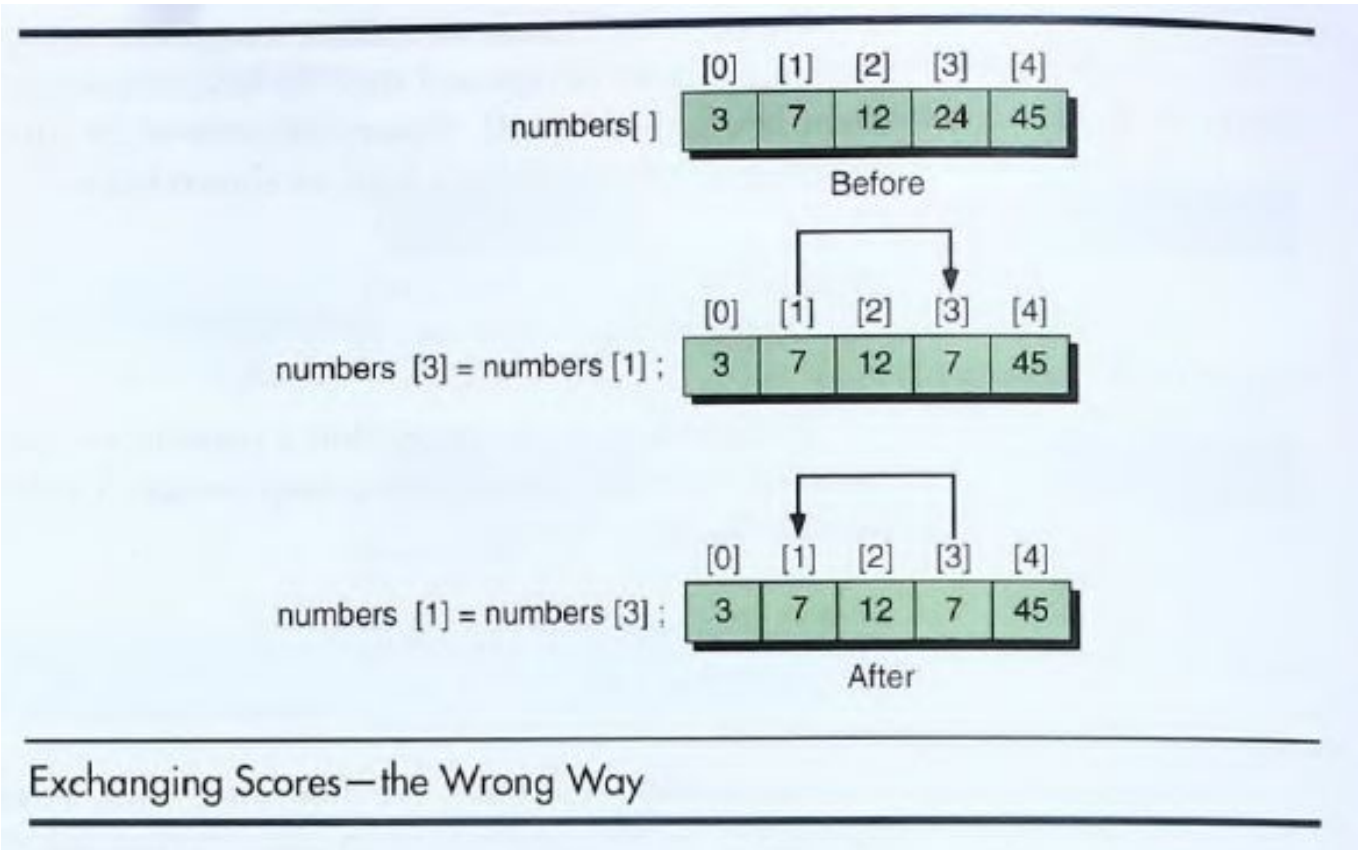
Unit-3

Initializes of Arrays in C



Unit-3

Exchanging Scores



Exchanging Scores—the Wrong Way

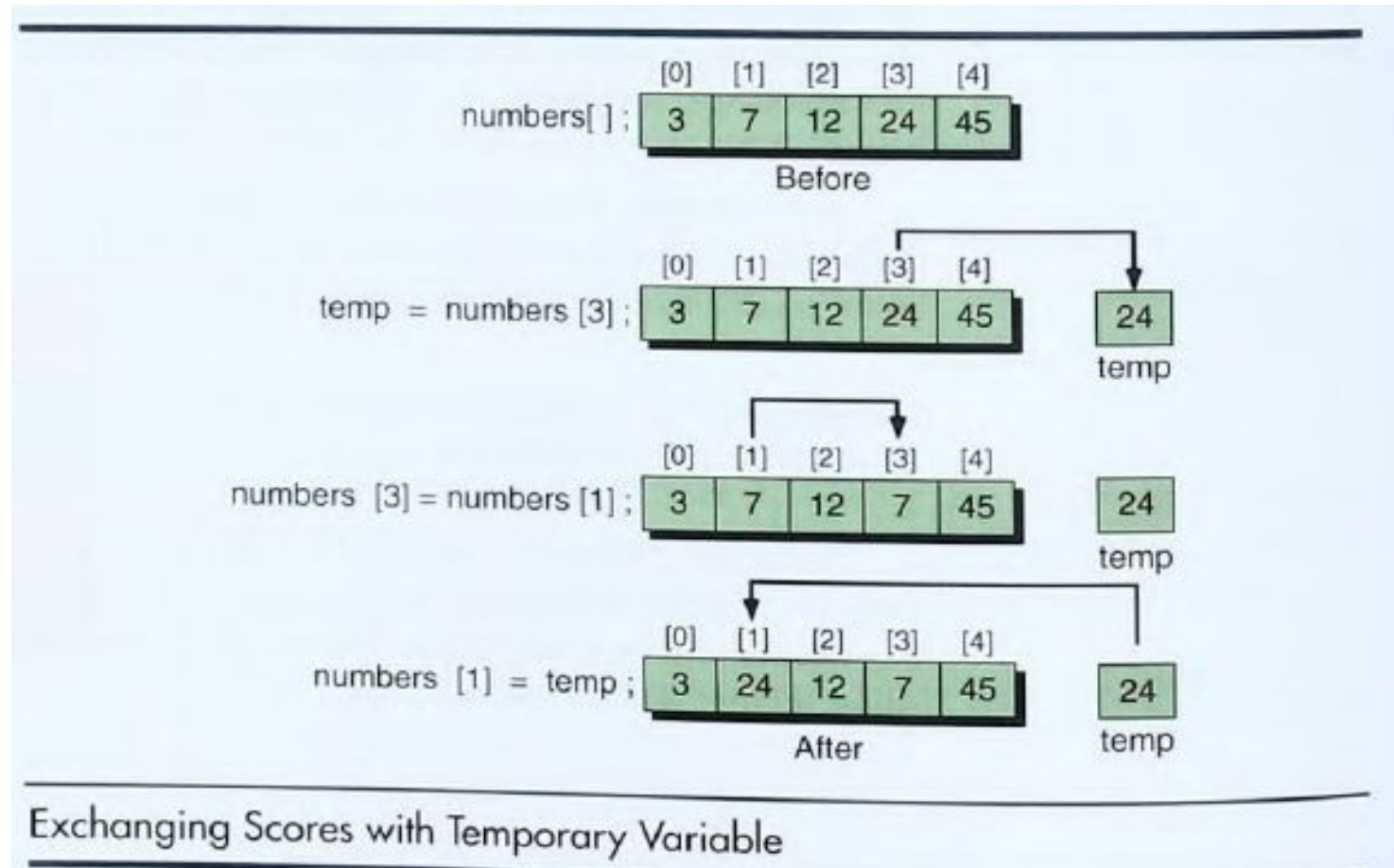
Exchanging Scores

Figure 8-9 shows that the original value of `numbers[3]` is destroyed before we can move it. The solution is to use a temporary variable to store the value in `numbers[3]` before moving the data from `numbers[1]`.

```
temp      = numbers[3];  
numbers [3] = numbers [1];  
numbers [1] = temp;
```

Unit-3

Exchanging Scores



- Types of Array :

1. One-dimensional arrays
2. Two-dimensional arrays
3. Multidimensional arrays

Unit-3

Accessing Array Elements

```
// C Program to illustrate element access using array
```

```
// subscript
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // array declaration and initialization
```

```
    int arr[5] = { 15, 25, 35, 45, 55 };
```

```
    // accessing element at index 2 i.e 3rd element
```

```
    printf("Element at arr[2]: %d\n", arr[2]);
```

```
    // accessing element at index 4 i.e last element
```

```
    printf("Element at arr[4]: %d\n", arr[4]);
```

```
    // accessing element at index 0 i.e first element
```

```
    printf("Element at arr[0]: %d", arr[0]);
```

```
    return 0;
```

Unit-3

```
// C Program to demonstrate the use of array
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // array declaration and initialization
```

```
    int arr[5] = { 10, 20, 30, 40, 50 };
```

```
    // modifying element at index 2
```

```
    arr[2] = 100;
```

```
    // traversing array using for loop
```

```
    printf("Elements in Array: ");
```

```
    for (int i = 0; i < 5; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Unit-3 One Dimensional Array

```
// Program to take 5 values from the user and store them in an array
// Print the elements stored in the array
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int values[5];
```

```
    printf("Enter 5 integers: ");
```

```
    // taking input and storing it in an array
```

```
    for(int i = 0; i < 5; i++) {
        scanf("%d", &values[i]);
    }
```

```
    printf("Displaying integers: ");
```

```
    // printing elements of the array
```

```
    for(int i = 0; i < 5; i++) {
        printf("%d\n", values[i]);
    }
```

```
    return 0;
```

```
}
```

1. Write a program to input the values into an array and print them in reverse order.
2. Write a program to print the index element and square of index

Unit-3 One Dimensional Array

```
#include <stdio.h>
{
    int i, n, a[100];
    printf("\n\nRead n number of values in an array and display it in reverse
order:\n");

    printf("Input the number of elements to store in the array :");
    scanf("%d", &n);
    printf("Input %d number of elements in the array :\n", n);
    for (i = 0; i < n; i++)
    {
        printf("element - %d : ", i);
        scanf("%d", &a[i]);
    }
    printf("\nThe values stored in the array are : \n");
    for (i = 0; i < n; i++)
    {
        printf("% 5d", a[i]);
    }
    printf("\n\nThe values stored in the array in reverse are :\n");
    for (i = n - 1; i >= 0; i--)
    {
        printf("% 5d", a[i]);
    }
}
```

Squares Array

```
1  /* Initialize array with square of index and print it.
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6  #define ARY_SIZE 5
7
8  int main (void)
9  {
10     // Local Declarations
11     int sqrAry[ARY_SIZE];
12
13     // Statements
14     for (int i = 0; i < ARY_SIZE; i++)
15         sqrAry[i] = i * i;
16
17     printf("Element\tSquare\n");
18     printf("=====\t=====\n");
19     for (int i = 0; i < ARY_SIZE; i++)
20         printf("%5d\t%4d\n", i, sqrAry[i]);
21     return 0;
22 }
```

Results:

Element Square

=====

0	0
1	1
2	4
3	9
4	16

Unit-3

One Dimensional Array

// C Program to illustrate strings

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // creating array of character
```

```
    char arr[6] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

```
    // printing string
```

```
    int i = 0;
```

```
    while (arr[i]) {
```

```
        printf("%c", arr[i++]);
```

```
    }
```

```
    return 0;
```

```
}
```

Unit-3

// C Program to illustrate the use of 1D array

#include <stdio.h>

int main()

{

// 1d array declaration

int arr[5];

// 1d array initialization using for loop

for (int i = 0; i < 5; i++) {

arr[i] = i * i - 2 * i + 1;

}

printf("Elements of Array: ");

// printing 1d array by traversing using for loop

for (int i = 0; i < 5; i++) {

printf("%d ", arr[i]);

}

return 0;

}

Unit-3

Program to search an element in an array

```
#include <stdio.h>
#define MAX_SIZE 100 // Maximum array size
int main()
{
    int arr[MAX_SIZE];
    int size, i, toSearch, found;
    printf("Enter size of array: ");
    scanf("%d", &size);
    printf("Enter elements in array: ");
    for(i=0; i<size; i++)
    {
        scanf("%d", &arr[i]);
    }
    printf("\nEnter element to search: ");
    scanf("%d", &toSearch);
    found = 0;
    for(i=0; i<size; i++)
    {
        if(arr[i] == toSearch)
        {
            found = 1;
            break;
        }
    }
}
```

```
if(found == 1)
{
    printf("\n%d is found at position %d", toSearch, i + 1);
}
else
{
    printf("\n%d is not found in the array", toSearch);
}

return 0;
}
```

Sorting and Searching:

Unit-3--- Bubble Sort

```
#include <stdio.h> int main()
{
int array[100], n, i, j, swap;
printf("Enter number of elements\n");
scanf("%d", &n);
printf("Enter %d integers\n", n);
for (i = 0; i < n; i++)
scanf("%d", &array[i]);
for (i = 0 ;i < n - 1; i++)
{
for (j = 0 ;j < n - i - 1; j++)
{
if (array[j] > array[j+1])
{
swap = array[j]; array[j] = array[j+1]; array[j+1] = swap;
}
}
}
printf("Sorted list in ascending order:\n"); for (i = 0; i < n; i++)
printf("%d\n", array[i]); return 0;
}
```

Unit-3---Binary Search

```
#include <stdio.h>
int binarySearch(int array[], int x, int low, int high)
{
// Repeat until the pointers low and high meet each other while (low <= high)
{
int mid = low + (high - low) / 2; if (array[mid] == x)
return mid; if (array[mid] < x)
low = mid + 1;
else
}
high = mid - 1;
return -1;
}
int main(void)
{
int array[] = {3, 4, 5, 6, 7, 8, 9};
int n = sizeof(array) / sizeof(array[0]); int x = 4;
int result = binarySearch(array, x, 0, n - 1); if (result == -1)
printf("Not found");
else
printf("Element is found at index %d", result);
return 0;
}
```


Unit-3---Two Dimensional Array—Matrix Addition

```
#include <stdio.h>
int main()
{
    int m, n, c, d, first[10][10], second[10][10], sum[10][10];
    printf("Enter the number of rows and columns of matrix\n"); scanf("%d%d", &m, &n);
    printf("Enter the elements of first matrix\n");

    for (c = 0; c < m; c++)
        for (d = 0; d < n; d++)
            scanf("%d", &first[c][d]);
    printf("Enter the elements of second matrix\n");
    for (c = 0; c < m; c++)
        for (d = 0; d < n; d++)
            scanf("%d", &second[c][d]);
    printf("Sum of entered matrices:-\n");
    for (c = 0; c < m; c++) {
        for (d = 0; d < n; d++) {
            sum[c][d] = first[c][d] + second[c][d];
            printf("%d\t", sum[c][d]);
        }
        printf("\n");
    }
    return 0;
}
```